Proyecto final · Bootcamp Full Stack

# Acompáñame

## Cuidadores de confianza, cuando tu familia más lo necesita

Andrea Vallina González · Factoría F5 · Octubre 2026



EL PROBLEMA

# Cuidar es difícil cuando no sabes a quién llamar

### Las familias

Buscan ayuda para cuidar a un familiar mayor y no saben en quién confiar: perfiles dispersos, sin valoraciones ni garantías.

### Los cuidadores

Tienen experiencia y vocación, pero no un lugar donde mostrarla, fijar su tarifa y recibir solicitudes ordenadas.

### El hueco

No hay un punto de encuentro sencillo, seguro y con reseñas reales entre quien necesita cuidados y quien los ofrece.

**Nunca sola, nunca solo.**

LA SOLUCIÓN

# Un marketplace con tres roles

### FAMILIA

Busca cuidadores, envía solicitudes, paga con tarjeta de prueba y valora el servicio.

### CUIDADOR

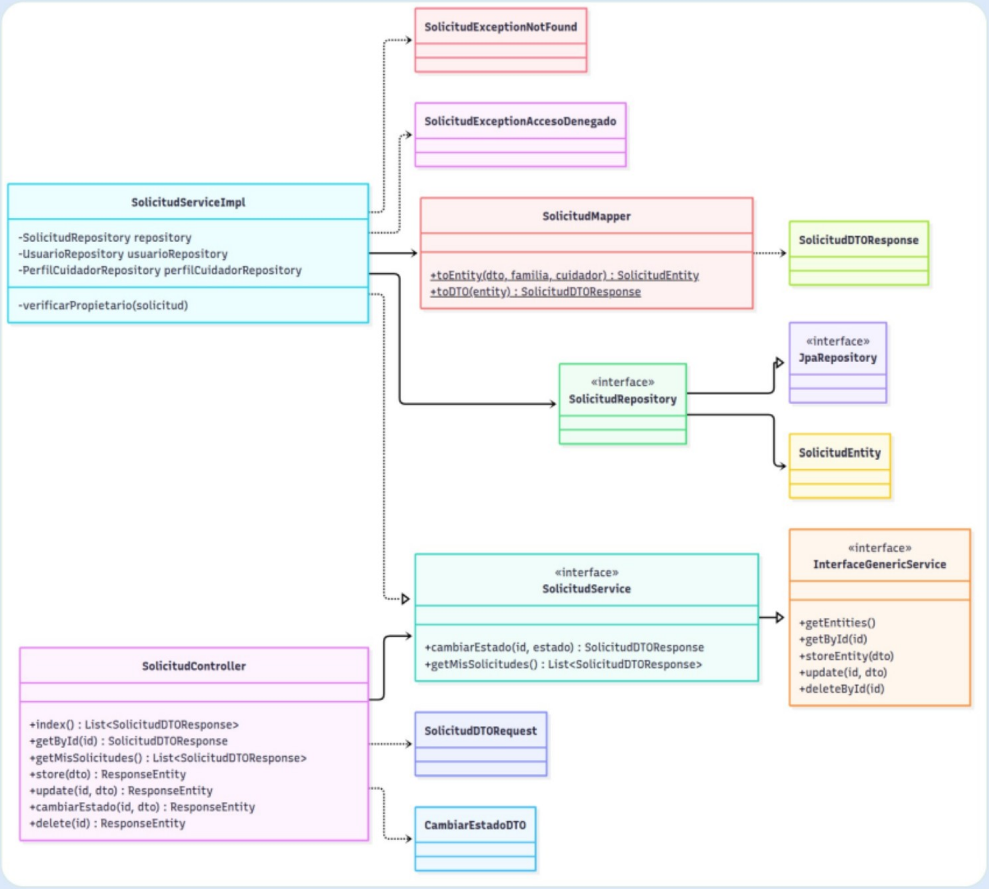
Gestiona su perfil y tarifa, acepta o rechaza solicitudes y recibe valoraciones.

### ADMINISTRACIÓN

Observa la plataforma y modera: estadísticas, avisos, usuarios y valoraciones. Nuevo.

ARQUITECTURA

# Arquitectura en capas




**Controller** recibe la petición y valida los datos.

**Service** contiene las reglas de negocio.

**Repository** accede a MySQL con Spring Data JPA.

**DTOs y Mappers** separan la API del modelo de datos.

Todo ello en Spring Boot 4 y Java 21, con MySQL en Docker. El paquete **admin** sigue las mismas capas.

MODELO DE DATOS

# Pocas tablas, bien relacionadas

### Usuario

Email, contraseña cifrada, rol y el campo activo, que permite bloquear sin borrar.

### Roles en tabla

FAMILIA, CUIDADOR y ADMIN viven en una tabla, no en un enum.

### Solicitudes

Unen a una familia con un cuidador y recorren sus estados.

### Valoraciones

Reseñas de las familias; el administrador puede eliminar las inadecuadas.

### Bloquear no es borrar

Un usuario bloqueado conserva su historial, pero deja de poder entrar. Se puede desbloquear en cualquier momento.

SEGURIDAD

# Cuatro capas de protección

### Autenticación

HTTP Basic con contraseñas cifradas.

### Autorización por rol

Cada ruta exige su rol; /api/v1/admin/** exige ADMIN.

### Protección IDOR

Cada usuario solo toca sus propios recursos.

### CORS

Solo orígenes permitidos.

**Un usuario bloqueado recibe 401 como con una contraseña incorrecta, y el efecto es inmediato.**

**Siguiente paso: migrar de Basic Auth a JWT.**

DISEÑO

# Del boceto al prototipo




1 · Boceto

2 · Mockup

3 · Aplicación final

**1** **Bocetos**

**2** **Wireframes**

**3** **Mockups**

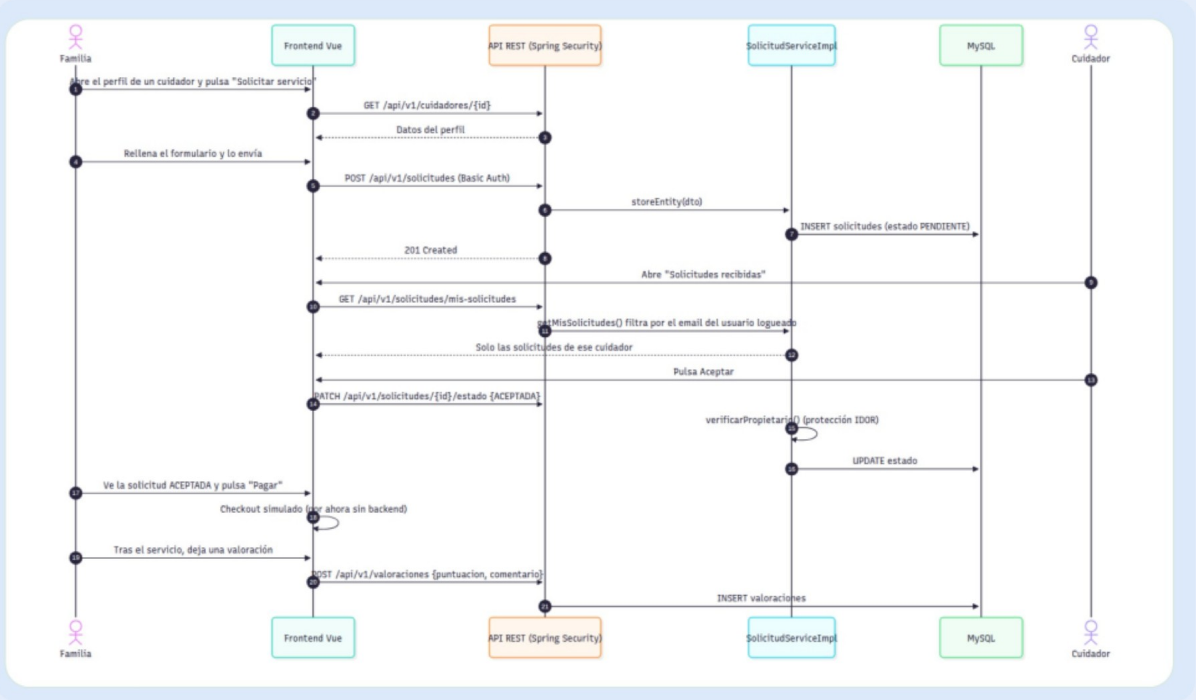
**4** **Prototipo en Lovable**

**5** **Figma**

Después, el panel de administración se diseñó directamente en código, con la misma guía de estilo.

FLUJO PRINCIPAL

# Así funciona una solicitud




1. La familia elige un cuidador.
2. Crea la solicitud, que queda pendiente.
3. El cuidador la acepta o la rechaza.
4. La familia paga con tarjeta de prueba.
5. Tras el servicio, deja su valoración.

**Cada paso valida el rol y la propiedad del recurso antes de cambiar nada.**

FRONTEND

# Frontend en Vue 3

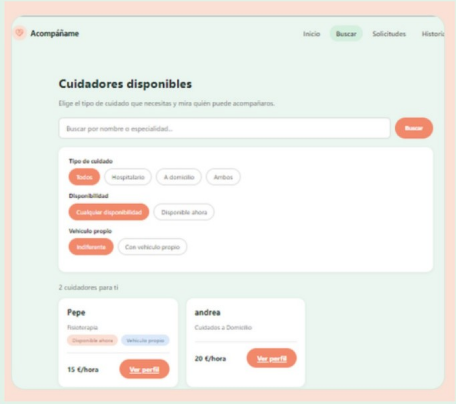
- Vue 3 con Composition API y Pinia
- Vue Router con guardas por rol
- Capa de repositorios para hablar con la API
- Componentes reutilizables y estilos BEM con variables CSS
- Diseño adaptable a móvil y escritorio

**19**

pantallas, incluido el panel de administración

**470**

tests unitarios más 9 de extremo a extremo



NOVEDAD

# Panel de administración

## Estadísticas y avisos

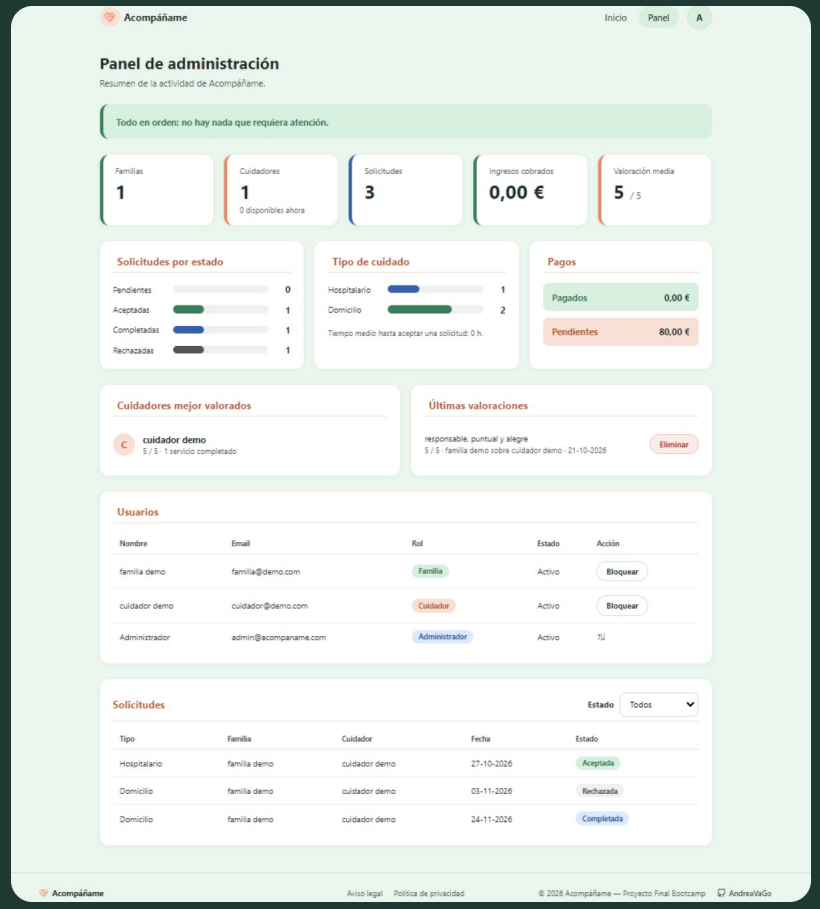
Usuarios, solicitudes y valoraciones de un vistazo, con avisos de lo que requiere atención.

## Usuarios

Lista completa con botón para bloquear o desbloquear.

## Valoraciones y solicitudes

Últimas reseñas, cuidadores mejor valorados y solicitudes filtradas por estado.







Escritorio y móvil

REGLAS DEL PANEL

# Moderar con límites claros

### Solo observa y modera

El administrador no crea ni edita solicitudes: solo consulta datos y retira contenido inadecuado.

### Dos acciones

Eliminar una valoración, con confirmación previa. Bloquear o desbloquear a un usuario.

### Sin autobloqueo

El administrador no puede bloquearse a sí mismo: el servidor responde 403.

**Todo bajo /api/v1/admin/**, accesible solo con el rol ADMIN.**

CALIDAD

# Probar antes de entregar

**436**

tests en el backend, con cobertura JaCoCo del 96 % en líneas y 95 % en ramas.

**470 + 9**

tests unitarios y de extremo a extremo en el frontend, con 98 % de cobertura.

**60**

tests propios del panel de administración, entre componentes, store y vistas.

**Casos límite cubiertos: usuario bloqueado, autobloqueo, rol incorrecto y recursos ajenos.**

LECCIÓN APRENDIDA

# Un test verde no sustituye al arranque real

### Qué pasó

Al arrancar la aplicación con el administrador, falló con LazyInitializationException al crear su usuario inicial.

### Por qué no se vio

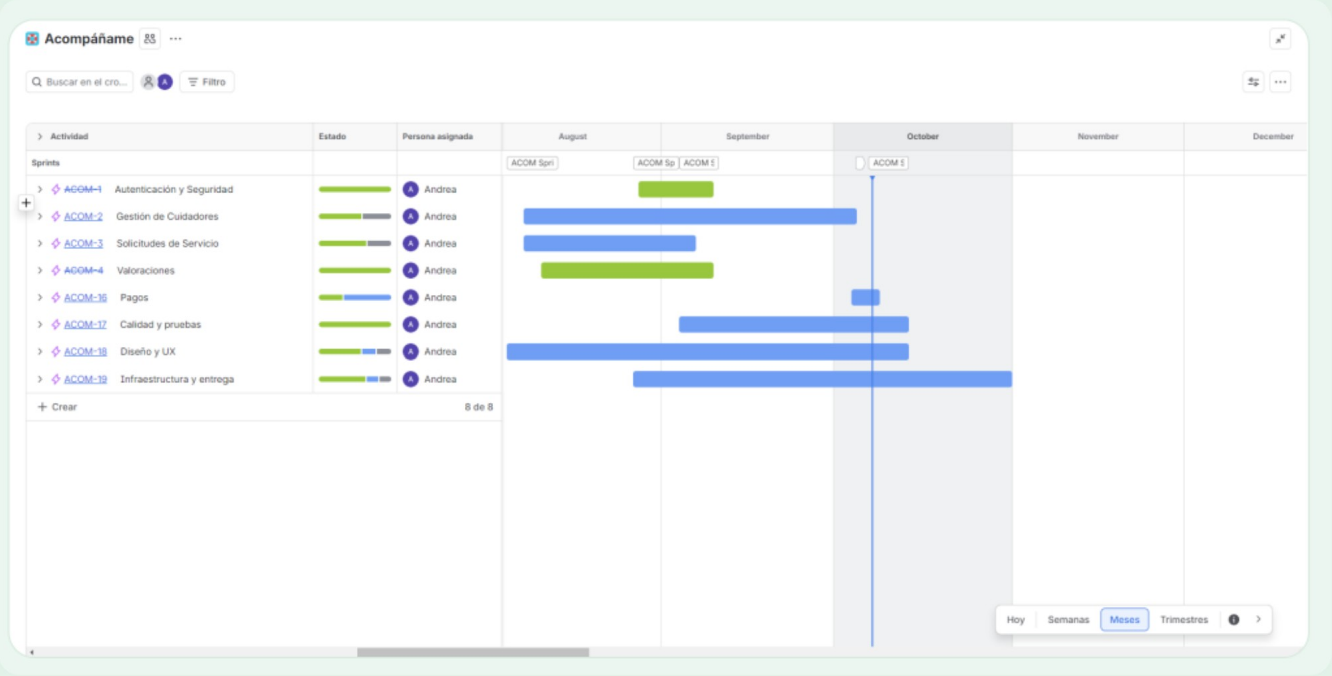
Los tests no recorrían el arranque real con las variables de entorno del administrador.

### Qué aprendí

Ejecutar la transacción completa en el inicializador y probar siempre un arranque manual antes de entregar.

METODOLOGÍA

# Scrum con Jira




**Cada commit del repositorio enlaza con su tarea de Jira, por ejemplo (ACOM-148).**

## 9 épicas

Cada bloque grande del proyecto, incluido el panel de administración.

## 19 historias

Descritas desde el punto de vista del usuario.

## 120 tareas

Repartidas en 6 sprints, con tablero y seguimiento.

DECISIONES

# Qué decidí y por qué

| | |
|---|---|
| **Patrón Repository** | Aísla la API del acceso a datos |
| **Roles en tabla** | Permite añadir roles sin tocar código |
| **hasAuthority frente a hasRole** | Sin prefijos ocultos |
| **DTOs y Mappers** | No se expone el modelo interno |
| **BEM y variables CSS** | Estilos coherentes y mantenibles |
| **Bloquear en vez de borrar** | Se conserva el historial |
| **Tarjeta de prueba en el checkout** | Sin manejar datos reales de pago |

FUTURO

# Qué mejoraría con más tiempo

## Próximos pasos

Migrar a JWT, usar una pasarela de pago real en modo de pruebas, alojar las fotos fuera del servidor y desplegar la aplicación.

## Fase 2

Chat, geolocalización, videollamadas, recordatorios e informes en PDF.

# Gracias

## ¿Preguntas?

github.com/AndreaVaGo/acompaname-backend

github.com/AndreaVaGo/acompaname-frontend

linkedin.com/in/andrea-vallina-gonzalez

